

Applied Implicit Computational Complexity

Extended Abstract

Neea Rusch

nrsch@augusta.edu

Augusta University

Augusta, Georgia, United States

Abstract

Implicit computational complexity (ICC) complements classic complexity theory by developing machine-independent characterizations of complexity classes. The idea is to introduce a restriction, at the level of a programming language, that guarantees every program satisfying the restriction belongs to a particular complexity class. There are several strong motivations for this approach, including that ICC systems drive better understanding of complexity classes, yield natural definitions and proofs, and produce techniques for automatable program analysis for free. However, despite the numerous advantages, ICC remains largely a theoretical novelty. This means the practical power, limitations, and advantages of ICC-based program analyses are not well-understood. The dissertation research “puts ICC theories to test” by investigating their capabilities outside the theoretical domain. A guiding intuition is that, if applied, implicit computational complexity could provide us new techniques for program analysis and verification.

Over a series of projects, the dissertation work yields three main findings. The first shows that ICC offers complementary techniques for analyzing resource usage. In other words, it allows to produce quantitative reports about programs execution behaviors in cases that are overlooked by other techniques. Second, it is possible to adjust the analyses to track other program properties. This suggests that unrealized potential is available in ICC systems, and we should learn to leverage it. The third finding is reflective and visionary. An essential feature of ICC is that it enables guaranteeing program properties by construction, before any program exists. This enables providing behavioral guarantees without needing to run any post-analysis. This is a powerful concept in principle, but still beyond reach. The practical attainability of this goal depends crucially on a viewpoint shift – we should consider ICC in the broader context it offers and continue exploring its applied potential.

CCS Concepts

• **Theory of computation** → **Program verification; Complexity theory and logic**; • **Software and its engineering** → **Formal software verification**.

Keywords

automatic resource analysis, implicit computational complexity, formal verification, static program analysis

1 Seeking Strong Software Quality Guarantees

The gold standard in formal verification is showing that an implementation of a program is totally correct. Total correctness requires proving two conditions about the program: that the program meets its specification, and that the computation terminates [24]. The

Implicit complexity in a nutshell. The goal of implicit computational complexity can be summarized as follows. Let $\mathcal{L}$ be a programming language, $\mathcal{C}$ a complexity class, and $\llbracket p \rrbracket$ the function computed by program p . Then, the task is to find a restriction $\mathcal{R} \subseteq \mathcal{L}$, such that the following equality holds:

$$\{\llbracket p \rrbracket \mid p \in \mathcal{R}\} = \mathcal{C}.$$

The variables $\mathcal{L}$, $\mathcal{C}$, and $\mathcal{R}$ are the parameters that vary greatly between different ICC systems [28, p. 7].

Figure 1: A general description of the systems of implicit computational complexity.

former ensures the program computes the expected output for all inputs. The latter ensures that the computed result will become observable eventually. Thus, a proof of total correctness gives us strong assurance that the program behaves expectedly at runtime, i.e., when the program is executed.

An additional consideration in reasoning about program correctness concerns non-functional properties. Non-functional properties characterize the program’s quality features like execution latency, power consumption, and information flow security [34]. Pure functional correctness is *practically unsatisfactory* if a program requires exorbitant time to compute its result; aggressively drains a device battery, or reveals secret information [3, 18]. Engineering truly high-quality programs thus requires additional techniques that enable us to formally verify non-functional properties. Unfortunately, analyzing non-functional properties is non-trivial and generally undecidable [29]. Developing specification formalisms and reasoning methods, that can provide some guarantees despite the undecidability, is an active area of research [21].

The field of *Implicit Computational Complexity (ICC)* [12]—cf. Figure 1—offers a conceivable pathway toward formal verification of certain non-functional properties. ICC is complementary to the classic complexity theory because it uses programming languages as the mechanism to reason about runtime behaviors. Broadly, complexity theory is concerned with resource requirements of computations [15]. Those requirements are organized in a hierarchy of complexity classes. The classes describe resource requirements w.r.t. an abstract (mathematical) machine model. The classes correspond roughly to resources like running time and memory consumption of programs that are executed on real-world computers. To streamline this reasoning, *implicit* computational complexity aims to discover *machine-independent* characterizations of the complexity classes. ICC systems are designed by introducing a *restriction*, at the level of a programming language, that guarantees a complexity property.

Then, every program satisfying the restriction is known to belong to a particular complexity class. This in turn provides a guarantee about the program’s runtime resource consumption.

Exchanging machine models for syntactic criteria is powerful in several ways. For example, it facilitates formal verification of programs. ICC enables defining syntactical specifications of resource consumption, and developing fully automatable analyses to verify whether a program satisfies such a specification [18]. ICC systems have been designed for numerous combinations of programming languages (imperative, lambda-calculus, function algebras, quantum, term rewrite systems, π -calculus, ...), restriction-techniques (linear logic, data-flow analysis, type systems, category theory, ...), and complexity classes (P , $PSPACE$, L , PP , ...) [26, 28].

2 Addressed Problem and Goals

Besides uses in formal verification, there are many good motivations for implicit reasoning about computational complexity. ICC allows guaranteeing program properties *by construction*, before any program exists. ICC drives better understanding of complexity classes and yields more natural definitions and proofs of central results than classical complexity theory [23]. For example, ICC systems show that complexity classes are intrinsic mathematical entities that do not depend on a particular machine model. Multiple ICC systems, e.g., [20, 25], are rich enough to express natural algorithms. Since the reasoning techniques differ from the existing alternatives, the results ICC systems produce are often orthogonal [3]. Finally, many ICC systems—e.g., [2, 17, 20, 25]—are *compositional*. This means they use modular techniques that enable bottom-up reasoning about subprograms. Compositionality is critical for scalability [11] and useful for proving soundness [22], i.e., that the analysis technique produces correct results. However, compositionality is missed by many prominent program analysis techniques [11, 33].

Despite this rich inventory of compelling features, ICC has remained largely a theoretical novelty, with a few exceptions [8, 9, 13, 16, 19, 27]. A theory in absence of applications means the power, limitations, and utilities of the theory are difficult to recognize. In our view [10, p. xxxv] [26, p. 75], theory and application are symbiotic. Investigations of both are needed for scientific advancement.

The embedded manifesto of the dissertation research is that *applications are necessary* to push forward our collective understanding of implicit computational complexity. It is conceivable that potentially great power is locked inside the theoretical ICC systems. It remains an open problem to release that power. Within the ICC community, more investigations are needed to explore the capabilities of the existing systems. Externally, the ICC-based program analyses should be exposed to broader research communities. These steps would motivate enhancements in ICC systems and facilitate discovery of new applications.

The work in the dissertation is only a small step toward these ambitious long-term goals. The realistic expectations of the dissertation are about technical and social advancement. Overall, there are four high-level goals.

- (1) Extend applied capabilities of ICC in automatic program analysis and verification.
- (2) Take ICC a few steps closer to integration into real-world software development workflows.

- (3) Initiate conversations on the relevance of ICC applications.
- (4) Expose ICC to broader research communities.

A guiding intuition behind these goals is that, if applied, implicit computational complexity could provide us several new practical program analyses. The use cases of such analysis include formal verification of many non-functional properties.

3 Research Questions and Approaches

The following *main hypothesis* guides the dissertation investigation.

Implicit computational complexity offers applied utilities when lifted from the theoretical domain.

Since the hypothesis is broad, the research projects that challenge it must be more refined. We focus on two paths. The first applies ICC in automatic analysis of *resource consumption*. This application is natural because it parallels the theoretical use of ICC. The second path takes the investigation to a more unconventional direction. It asks whether we can adjust ICC systems to track *other* program properties, beyond resource consumption. Stated differently, by swapping the property of interest, it allows us to assess the technical flexibility of ICC systems. To summarize, we investigate whether we can (i) apply the systems in the originally designed ways, and (ii) discover unexpected applications.

3.1 Automatic Resource Analysis

This series of work focuses on an ICC system called the *flow calculus of mwp-bounds* [20]. The flow calculus is a canonical example of a theoretical ICC system. It focuses on tracking change in data size during computation. For imperative programs, it aims to determine if it is possible to bound the growth of variable values by polynomials in inputs. The choice to focus on this particular ICC system is based on intuition. As a syntactical and compositional data-flow analysis, it seems like a good candidate for automatic resource analysis.

Several research questions arise from the flow calculus.

- (RQ1) Can we develop an *automatic* program analysis based on the theory?
- (RQ2) Given its paper proofs, is the theory correct, i.e., can we prove formally the soundness of the flow calculus?
- (RQ3) Assuming the theory can be automated, what are the use cases of such analysis?

These questions are still conventional because they target resource analysis. In other words, given a complexity-theoretic technique, we should reasonably expect that it can be applied in the designed way. If successful, the investigation should minimally produce two research artifacts—a program analyzers and mechanized proof—corresponding to RQ1 and RQ2. Additionally, it should produce new insights of the flow calculus and its possible applications. Since ICC is not the only technique for analyzing growth in data size, having an automatic analyzer would enable comparing it to the alternative approaches.

3.2 Analyzing Extended Properties

The second series of work applies implicit computational complexity to track other non-functional program properties. The idea is

to select an ICC system and adjust it to capture a different non-functional property. The motivation is multifaceted. It requires developing deep understanding of the mechanics of the chosen technique. Conceptually, it pushes to think about ICC outside the usual frame of computational complexity. Lastly, investigations in this direction are important to expose ICC to broader research communities, since the application is shifted away from (or beyond) resource consumption.

In this series of work, the starting point is an ICC system based on quasi-invariants (QI), as formulated in [27]. Similar to the flow calculus, the QI framework is a data-flow analysis of imperative programs. However, the QI framework supports a richer input language and is founded on a seemingly adjustable mathematical framework. In the prior formulation [27], it was used to obtain a compile-time program optimization. The optimization lifts nested loops outside their containing loop, such that post-transformation the program complexity is reduced.

We ask the following questions about the extended uses of the QI framework.

- (RQ4) How to develop a program transformation to increase parallelization potential?
- (RQ5) How to use it to analyze security properties, specifically non-interference?

Research questions 4 and 5 are posed here with an air of obviousness. However, arriving to those questions was not obvious. A substantial hidden challenge of the dissertation research plan involves identifying suitable research domains—like parallel programming or language-based security [32]—where ICC-based techniques could offer meaningful benefit.

Another consideration is that adjusting an ICC system may lead to losing the original guarantee about resource consumption. In return, we gain an alternative guarantee about program behavior. Whether this trade is beneficial depends on the application. Therefore, besides tracking properties, this investigation is also about studying the mechanics of ICC systems, and uncovering how exactly they provide behavioral guarantees.

Finally, this direction of research contains a bonus-challenge. Simply showing that an ICC-based technique can solve a problem in another application domain is not enough. It is necessary to show that the solution is *independently* relevant in the target domain, regardless of the origins of the solution. Based on experience, this criterion is strongly enforced by the scientific peer review process.

4 Methodology and Research Deliverables

Following the approaches of section 3, we conducted a series of investigations. These resulted in several publications [3, 6, 7], workshop presentations [4, 5, 30], and ongoing work. The dependence associations of the written works are illustrated in Figure 2. We can summarize the main contributions as follows.

For RQ1, we enhanced the theory to develop an automatic resource analysis in [3, 6, 31]. Due to inherent nondeterminism, developing an efficient and useful practical analysis was surprisingly difficult, but eventually achievable. The enhanced flow calculus can answer the same questions as the original formulation, but automatically and in practice. Moreover, through the enhancements we discovered how to make the calculus more expressive, allowing it

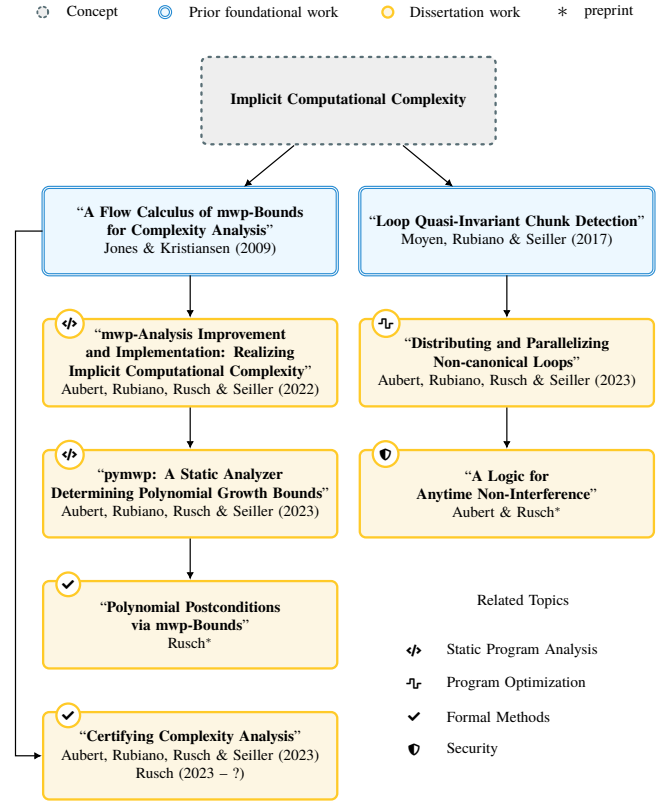


Figure 2: Research publications and their associations.

to cover a larger class of programs than the original theory [31]. The flow calculus is implemented as an open source static analyzer, *pymwp*¹² for analyzing a subset of C programs [6]. Based on this work, we discovered that the analysis can support inference of specification conditions [31] that are needed for formal verification. This finding corresponds to RQ3. Proving the soundness of the flow calculus (RQ2) is still ongoing work [7]. However, given the other developments, a formalization effort is now even more strongly motivated and justified.

Along the second direction, for RQ4, we applied the QI framework to develop a program optimization for loop parallelization [7]. This is useful, because the optimization can be applied to loops whose iteration count is unknown. Parallelizing such loops is outside the capabilities of prior techniques. In an ongoing investigation, for RQ5, we apply the QI framework to tracking programs non-interference [14], a kind of confidentiality property. Non-interference ensures a program does not reveal secrets; and if it does, we can identify where the violation occurs. Our working intuition is that, since the analysis can be adjusted to different programming languages, it enables tracking *preservation* of the non-interference property at different compilation stages, while a program is transformed from source code to an executable. This is different from

¹Pronounced *pat em double-you pē*.

²<https://statycc.github.io/pymwp/>

many existing techniques that analyze programs only at one stage of representation.

5 Research Findings and Conclusions

On automatic resource analysis. We can now confirm that it is possible to obtain automatic program analysis from the flow calculus of mwp-bounds, but only after substantial adjustments. An efficient program analysis required inventing ways to manage costly sub-computations and solving additional problems that were exposed in the process. At conclusion, the enhanced technique is not just automatable, but can analyze more programs than the original theory. We can now confirm the analysis results are complementary to alternative state-of-the-art techniques [6, p. 5]. Thus, it enables us to analyze different questions about resource consumption. The implemented flow calculus can also be used in specification inference. There are potentially many more applications of the flow calculus remaining to be discovered.

On analysis of extended program properties. The QI framework provides a case example showing that ICC systems can be flexible enough to track other program properties. While adjusting the system, we made a surprising discovery in that changing the property of interest simplified the mathematical analysis. For example, to track a complexity property, we must compute fixed points and preserve the program command order. In the adjusted analysis, it was possible to relax these conditions. More generally, that the QI framework can be used to track various properties is reminiscent of the Dependency Core Calculus (DCC) [1]. In DCC, different program properties are modeled as *instances* of a central notion of *dependency*. This mirrors our observations about the QI framework. This suggests that the QI framework encapsulates even wider utility than uncovered so far during this dissertation work.

On broader dissertation goals. Zooming out of the specific projects, we can reflect on the main hypothesis and the research goals. We have successfully gathered evidence of the application potential of ICC. The evidence supports the main hypothesis, but is still too limited to draw generalizing conclusions. However, our experiences of developing the ICC systems strongly reinforced the symbiotic view of theory and application. In the process, we faced many scientific and engineering challenges, and identified compelling motivations to justify the investigations. At conclusion, we have strengthened the theories of interest and clarified their practical relevance.

In section 2, we crafted four goals to define the dissertation expectations. The first was about extending the applied capabilities of ICC, which was met successfully. The second was about introducing ICC in development workflows. Although we have developed a static analyzer, it is a step in “isolation.” Integrating ICC analyses into other software development tools (compilers, verifiers, etc.) remains as an outstanding task. Such integration is important to promote relevance and continue development of ICC techniques. Our work in using the flow calculus to infer specification conditions [31] is a first step in that direction. The last two goals were social and about disseminating ideas across research communities. Within the ICC community, our efforts materialized in seminar talks and informal conversations. Although this record is below

ideal, the ideas in the published works will remain discoverable to others in written form, asynchronously and in perpetuity, like [26]. In regard to other communities, we were successful at exposing the work at verification, programming languages, and security venues. Thus, the fourth goal was satisfactorily met.

Final thoughts & future perspectives. The ICC approach of guaranteeing program properties demonstrates great applied potential. For example, it offers a pathway toward formal verification of many non-functional properties. One research direction, that deserves more attention, is applying ICC systems to ensure properties by construction. This would provide guarantees before any program exists and eliminates the need for *a posteriori* analysis. Although the dissertation research did not extend that far, exploring this direction is an intriguing future goal. Whether it is attainable depends crucially on a viewpoint shift. We should regard *implicit computational complexity* in the broader context it offers—including as a rich toolbox of techniques for static program analysis—and continue the exploration of its applied potential.

Acknowledgments

Many thanks to Clément Aubert for the years of conversations and collaborations that have guided and shaped this research. Also, many thanks to Thomas Rubiano and Thomas Seiller for their contributions in the associated publications.

References

- [1] Martín Abadi, Anindya Banerjee, Nevin Heintze, and Jon G. Riecke. 1999. A core calculus of dependency. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL99)*. ACM, 147–160. <https://doi.org/10.1145/292540.292555>
- [2] Robert Atkey. 2024. Polynomial Time and Dependent Types. *Proceedings of the ACM on Programming Languages* 8, POPL (1 2024), 2288–2317. <https://doi.org/10.1145/3632918>
- [3] Clément Aubert, Thomas Rubiano, Neea Rusch, and Thomas Seiller. 2022. mwp-Analysis Improvement and Implementation: Realizing Implicit Computational Complexity. In *7th International Conference on Formal Structures for Computation and Deduction (FSCD 2022) (LIPIcs, Vol. 228)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 26:1–26:23. <https://doi.org/10.4230/LIPIcs.FSCD.2022.26>
- [4] Clément Aubert, Thomas Rubiano, Neea Rusch, and Thomas Seiller. 2022. Realizing Implicit Computational Complexity. (6 2022), 4 pages. https://types22.inria.fr/files/2022/06/TYPES_2022_paper_14.pdf At the 28th International Conference on Types for Proofs and Programs (TYPES).
- [5] Clément Aubert, Thomas Rubiano, Neea Rusch, and Thomas Seiller. 2023. Certifying Complexity Analysis. (1 2023), 4 pages. <https://hal.science/hal-04083105v1/file/main.pdf> At the Ninth International Workshop on Coq for Programming Languages (CoqPL).
- [6] Clément Aubert, Thomas Rubiano, Neea Rusch, and Thomas Seiller. 2023. *pymwp: A Static Analyzer Determining Polynomial Growth Bounds*. Springer Nature Switzerland, 263–275. https://doi.org/10.1007/978-3-031-45332-8_14
- [7] Clément Aubert, Thomas Rubiano, Neea Rusch, and Thomas Seiller. 2023. *Distributing and Parallelizing Non-canonical Loops*. Springer Nature Switzerland, 1–24. https://doi.org/10.1007/978-3-031-24950-1_1
- [8] Martin Avanzini and Ugo Dal Lago. 2017. Automating Sized-Type Inference for Complexity Analysis. *Proceedings of the ACM on Programming Languages* 1, ICFP (8 2017), 43:1–43:29. <https://doi.org/10.1145/3110287>
- [9] Martin Avanzini, Georg Moser, and Andreas Schnabl. 2008. *Automated Implicit Computational Complexity Analysis (System Description)*. Springer Berlin Heidelberg, 132–138. https://doi.org/10.1007/978-3-540-71070-7_10
- [10] Matt Bishop. 2019. *Computer Security: Art and Science*. Addison-Wesley Professional.
- [11] Quentin Carbonneaux, Jan Hoffmann, and Zhong Shao. 2015. Compositional Certified Resource Bounds. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ’15)*. ACM, 467–478. <https://doi.org/10.1145/2737924.2737955>
- [12] Ugo Dal Lago. 2011. *A Short Introduction to Implicit Computational Complexity*. LNCS, Vol. 7388. Springer Berlin Heidelberg, 89–109. https://doi.org/10.1007/978-3-642-31485-8_3

- [13] Hugo Férée, Samuel Hym, Micaela Mayero, Jean-Yves Moyen, and David Nowak. 2018. Formal proof of polynomial-time complexity with quasi-interpretations. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP '18)*. ACM, 146–157. <https://doi.org/10.1145/3167097>
- [14] Joseph A Goguen and José Meseguer. 1982. Security policies and security models. In *1982 IEEE Symposium on Security and Privacy*. IEEE, 11–20. <https://doi.org/10.1109/SP.1982.10014>
- [15] Oded Goldreich. 2008. *Computational complexity: a conceptual perspective*. Cambridge University Press.
- [16] Emmanuel Hainry, Emmanuel Jeandel, Romain Péchoux, and Olivier Zeyen. 2021. *Complexityparser: An automatic tool for certifying poly-time complexity of Java programs*. Springer International Publishing, 357–365. https://doi.org/10.1007/978-3-030-85315-0_20
- [17] Emmanuel Hainry and Romain Péchoux. 2023. A General Noninterference Policy for Polynomial Time. *Proceedings of the ACM on Programming Languages* 7, POPL (1 2023), 806–832. <https://doi.org/10.1145/3571221>
- [18] Sylvain Heraud and David Nowak. 2011. *A Formalization of Polytime Functions*. Springer Berlin Heidelberg, 119–134. https://doi.org/10.1007/978-3-642-22863-6_11
- [19] Jan Hoffmann, Klaus Aehlig, and Martin Hofmann. 2012. *Resource Aware ML*. Springer Berlin Heidelberg, 781–786. https://doi.org/10.1007/978-3-642-31424-7_64
- [20] Neil D. Jones and Lars Kristiansen. 2009. A flow calculus of mwp-bounds for complexity analysis. *ACM Transactions on Computational Logic* 10, 4 (8 2009), 28:1–28:41. <https://doi.org/10.1145/1555746.1555752>
- [21] Benjamin Kaminski. 2025. *Ina Schaefer on X-by-Construction*. Retrieved April 27, 2025 from <https://etaps.org/blog/031-ina-schaefer/> (ETAPS Blog).
- [22] Sven Keidel. 2021. *Modular Specification and Compositional Soundness of Abstract Interpreters*. PhD thesis. Johannes Gutenberg-Universität Mainz. <https://doi.org/10.25358/openscience-6384>
- [23] Lars Kristiansen. 2017. Slides of talk “On Resource Analysis of Imperative Programs”. <https://arxiv.org/html/1704.05169> DICE-FOPARA 2017, Uppsala, Sweden (Accessed: 2025-01-06).
- [24] K Rustan M. Leino. 2023. *Program proofs*. MIT Press.
- [25] Jean-Yves Marion. 2011. A Type System for Complexity Flow Analysis. In *2011 IEEE 26th Annual Symposium on Logic in Computer Science*. IEEE, 123–132. <https://doi.org/10.1109/lics.2011.41>
- [26] Jean-Yves Moyen. 2017. *Implicit Complexity in Theory and Practice*. Habilitation à Diriger des Recherches (HDR). University of Copenhagen. https://lipn.univ-paris13.fr/~moyen/papiers/Habilitation_JY_Moyen.pdf
- [27] Jean-Yves Moyen, Thomas Rubiano, and Thomas Seiller. 2017. *Loop Quasi-Invariant Chunk Detection*. Springer International Publishing, 91–108. https://doi.org/10.1007/978-3-319-68167-2_7
- [28] Romain Péchoux. 2020. *Complexité implicite : bilan et perspectives*. Habilitation à Diriger des Recherches (HDR). Université de Lorraine. <https://hal.univ-lorraine.fr/tel-02978986v3/file/HDR-RP.pdf>
- [29] Henry Gordon Rice. 1953. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical society* 74, 2 (3 1953), 358–366. <https://doi.org/10.1090/S0002-9947-1953-0053041-6>
- [30] Neea Rusch. 2022. Formally Verified Resource Bounds Through Implicit Computational Complexity. In *Companion Proceedings of the 2022 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '22)*. ACM, 17–20. <https://doi.org/10.1145/3563768.3565545>
- [31] Neea Rusch. 2025. Polynomial Postconditions via mwp-Bounds. In review.
- [32] Andrei Sabelfeld and Andrew C Myers. 2003. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications* 21, 1 (1 2003), 5–19. <https://doi.org/10.1109/JSAC.2002.806121>
- [33] Fabian Schiebel, Florian Sattler, Philipp Dominik Schubert, Sven Apel, and Eric Bodden. 2024. Scaling Interprocedural Static Data-Flow Analysis to Large C/C++ Applications: An Experience Report. In *38th European Conference on Object-Oriented Programming (ECOOP 2024) (LIPIcs, Vol. 313)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 36:1–36:28. <https://doi.org/10.4230/LIPIcs.ECOOP.2024.36>
- [34] Maurice H. ter Beek, Loek Cleophas, Ina Schaefer, and Bruce W. Watson. 2018. *X-by-Construction*. Springer International Publishing, 359–364. https://doi.org/10.1007/978-3-030-03418-4_21